Running an Agent Team on a Mixture-of-Experts Decode Kernel: An Experience Report from the Open Kernel Challenge

AUTHOR NAME
Independent researcher

November 5, 2026

Abstract

The Muse Spark Open Kernel Challenge scored entrants on one GPU's share of a Mixture-of-Experts transformer decoder layer, timed on an NVIDIA H200 against SGLang's production kernels across twelve shapes. Over nine days in late 2026 one person ran a team of LLM coding agents on this problem. Our own kernels reached 2.043× SGLang. We then forked the best public layer file, which measured 2.249× on our account. Our safety audit of the fork switched off several features whose ordering or progress we could not justify, and the audited fork scored 2.053×. The first public fork, with one change of ours, scored 2.126×, and three more days brought the final file to a mean of 2.250× over nine judged runs. Single-run steps attribute roughly half of those three days' gain to writing correctness arguments for disabled features and switching them back on: programmatic dependent launch across kernel boundaries and per-expert completion counters in place of a grid barrier. Only the first of those steps is clearly larger than the judge's run-to-run noise. The best run, 2.280×, ranked 153rd on the layer board late on the last day. We describe the layer and its timing rules, what we inherited and what we changed, the variants that lost, an organizer disqualification of a routing optimization we had read as allowed, and the process failures of an agent team working against a rationed judge.

1 Introduction

GPU kernel competitions now attract agent-driven entries. The Open Kernel Challenge [1] posed five problems drawn from one model, Muse Spark Open. One of them was the whole decoder layer for batched decode and speculative verification. Its board counted toward the overall board, which ranks teams by how much their best runs together would speed up a decode step.

This report covers one entry. The work ran from 27 October to the close of kernel submissions at midnight Pacific on 4 November 2026. One person steered. LLM coding agents wrote nearly all of the code, ran the experiments and kept the records. Most were Claude Code sessions. Codex sessions carried the forked line on 1 and 2 November, on a branch of their own. The repository holds about 2,650 commits, a ledger of every judged run and a list of about 445 ruled-out variants, most with their measurements [3].

The result is modest. The final file's mean speed matches the public file it descends from, and its best run sat about 1% below the 100th-ranked entry's score at the last reading of it, at 18:28 Pacific on the last day. We think the record is still worth publishing, for three reasons:

- It measures where time goes in a well-optimized MoE decode layer at small batch, and which plausible optimizations lose there.

- It gives a concrete case of a synchronization lesson. A grid-wide barrier in the routed-expert kernel could be replaced by per-expert completion counters once someone re-read the disabled code instead of treating it as settled. One hang risk stays recorded against the replacement.
- It records how an agent team failed: stale clocks, shared checkouts, a measurement flag that changed the thing measured, and a routing optimization that passed every numerical check and was then disqualified.

2 Related work

The baseline we were timed against is SGLang [8], which serves this layer with FlashAttention-3 [9] for attention, Marlin [10] for the 4-bit-weight expert GEMMs and cuBLAS for the dense projections. Our expert kernel assigns work with a stream-K decomposition [11]. MegaBlocks [12] and DeepGEMM [13] are public examples of grouped GEMM kernels for MoE layers, the second written for Hopper GPUs. The synchronization arguments in Section 5 rest on the PTX memory consistency model [15] and on CUDA's programmatic dependent launch [16].

On LLM agents for kernels, KernelBench [14] measures whether models can write correct and fast GPU kernels for isolated operators. Our setting differs in three ways: the target was a whole production layer, a public competition judge set the timing and correctness rules, and the agents worked for days as a team with shared state. The public entries we built on ([4, 5]) and EAT [6], the agent supervisor we used on the last day, are the closest prior work we know of. The keell0renz entry comes with notes on about 460 experiments; none of them is a paper.

3 The benchmark

3.1 What one step computes

The problem is everything one GPU does for one layer under 8-way tensor parallelism and 8-way expert parallelism, with all-reduces treated as the identity [2]. A step processes T tokens. With x the attention input ($T \times 8192$, BF16) and s the FP32 residual stream:

$$
\begin{aligned}
q, k, v, z &= \mathrm{split}(xW_{qkvg}^{\top});\ \text{QK-norm, RoPE (sliding layers only), write } \mathrm{fp8}(k), \mathrm{fp8}(v) \\
a &= \big(\mathrm{rmsnorm}(\mathrm{attn}(q)) \odot \sigma(z)\big) W_o^{\top} \\
s &\leftarrow \alpha_1 s + \beta_1\, \mathrm{rmsnorm}(a), \qquad h = \mathrm{rmsnorm}(s) \odot (1 + w_1) \\
E &= \mathrm{top8}\big(\sigma(0.177\, hW_r^{\top}) + b\big) \\
p &= \mathrm{rmsnorm}(hW_{pool}^{\top}) \odot (1 + w_2), \qquad y = \textstyle\sum_{e \in E \cap \mathrm{local}} g_e\, \mathrm{rmsnorm}\big(\mathrm{expert}_e(p) \odot (1 + w_3)\big) \\
f &= yW_{unpool}^{\top}, \qquad s \leftarrow \alpha_2 s + \beta_2\, \mathrm{rmsnorm}(f \odot (1 + w_4))
\end{aligned}
$$

Here σ is the logistic sigmoid, z is the attention output gate and "local" means the experts this GPU owns. α_i and β_i are fixed residual scales, w_i are learned norm weights, b is the router's bias and 0.177 is the router's fixed logit scale, all given by the model. g_e is expert e's unbiased sigmoid score divided by the sum of those scores over E. Up to $T = 8$ this GPU computes only its eighth of the unpooler's output columns. The kernel returns $\mathrm{rmsnorm}(s) \odot (1 + w_5)$, the new s and the chosen experts.

This GPU holds 16 query heads and 2 KV heads of dimension 64, so W_{qkvg} is 2304×8192 and W_o is 8192×1024. The router scores 256 experts in FP32, and this GPU owns 32 of them. A pooler projects to width 4096 before the experts and an unpooler projects back. Each expert is a

Table 1: The twelve timed shapes on the final file's best judged run (`908cc07e43364008`). Times in µs. The floor is the judge's speed-of-light bound.

load	layer	SGLang	ours	speedup	floor	ours/floor
decode 1	sliding	170.77	65.84	2.594	40.02	1.64
decode 1	full	180.74	71.66	2.522	44.18	1.62
decode 16	sliding	268.51	127.15	2.112	76.72	1.66
decode 16	full	439.92	202.29	2.175	137.46	1.47
decode 32	sliding	261.92	141.18	1.855	84.51	1.67
decode 32	full	621.41	292.72	2.123	208.10	1.41
verify 1	sliding	260.02	99.91	2.603	62.29	1.60
verify 1	full	281.15	113.22	2.483	66.44	1.70
verify 16	sliding	507.90	233.53	2.175	126.61	1.84
verify 16	full	672.82	311.93	2.157	188.68	1.65
verify 32	sliding	701.26	298.79	2.347	143.81	2.08
verify 32	full	1071.82	466.19	2.299	267.18	1.74

SwiGLU block at width 4096 whose weights are NVFP4: E2M1 values, one FP8 scale per 16 values and one FP32 scale per expert and matrix. Activations stay BF16 wherever SGLang keeps them. The KV cache is FP8 E4M3, and queries may be rounded to E4M3 as FlashAttention-3 does.

3.2 Shapes, timing and score

There are six loads: decode at batch 1, 16 and 32 ($T = 1, 16, 32$), and speculative verification of an 8-token draft at batch 1, 16 and 32 ($T = 8, 128, 256$). We call decode 1 and verify 1 the short loads. Each load times one sliding-window layer (window 2,048, weight 46) and one full-attention layer (weight 16), both at a context of about 80,000 tokens. A load's time is the weighted sum of its two shapes. The score is the geometric mean over loads of SGLang's time over ours, so each load counts equally. The board shows a time as well: SGLang's fixed per-step time of 22,620.3 µs divided by the speedup.

The judge captures the call as a CUDA graph and replays it. Every replay starts with a cold L2, and our replays alternate with SGLang's on the same fresh inputs. A shape's speedup is the median of paired ratios, with the order of each pair swapped so that which side runs first cancels. Each run reports a 95% interval of at least ±1.5%; the judge does not publish how it computes the interval. The baseline is SGLang at commit `a55ab7d5`. Before timing, a run must pass 13 correctness tests, some at contexts up to 2^{20} tokens, and every timed call is checked against an FP32 reference.

The rules matter as much as the math. Weights may be repacked losslessly but not stored at a lower precision. Every key in the window or context must be attended to. Nothing derived from the inputs may be cached. Routing may differ from the reference only on near-ties, within stated tolerances. A submission is a single Python file of at most 1 MB whose `prepare` function runs once, untimed, for up to 300 s; we call that phase preparation. Each problem allowed a daily number of public runs and of private "check" runs, and identical source submitted twice returns its earlier result.

Table 1 lists the twelve shapes with SGLang's time, our final file's time on its best run, and the speed-of-light floor. The floor is the time to move the data at 4.8 TB/s or to do the math at 989 TFLOP/s (dense BF16), whichever is longer.

Table 2: Software and hardware on the judge and on the rented box. The judge column comes from its run receipts, except nvcc, which the challenge's documentation names.

item	judge	rented box
GPU	H200, 700 W limit	H200 SXM, 132 SMs, 700 W limit
driver	580.159.03	595.91.07 (CUDA 13.2)
PyTorch	2.14.1+cu130	2.14.1+cu130, Triton 3.8.0
cuBLASLt	13.1.1	13.1.1
nvcc	13	13.0.88
SGLang	`a55ab7d5`	`a55ab7d5`

3.3 Environment

Table 2 lists what we know of the judge's machine and of the H200 we rented for paired comparisons, which we call the box. The judge's details come from its run receipts, which report clocks (1,980 MHz SM, 3,201 MHz memory) and an opaque node id but no host name. At first the box ran the short loads 9–18% faster than the judge, mostly inside the expert kernel. The cause was our box harness: it gave every fresh input the same choice of which experts count as local, while the judge draws that per input. With the harness fixed, box times at decode 1 and verify 1 rose by 11 to 22%, which covers the gap. The box's driver is also newer than the judge's.

3.4 Where the baseline spends its time

Profiles of the SGLang baseline show three regimes. At decode 1 on a sliding layer (170.0 µs), no single kernel dominates. The Marlin expert GEMMs take 40.8 µs (23%) and a split-K cuBLAS projection 31.4 µs (18%); the rest is a long chain of small launches. At decode 32 on a full layer (610.1 µs), FlashAttention-3 takes 68% reading the 80K-token FP8 cache for 32 requests. At verify 32 on a sliding layer (676.7 µs), Marlin takes 73%. A competitive entry needs all three: a short launch chain at $T = 1$, an attention kernel that streams the cache at close to full bandwidth, and an expert kernel that reads each active expert's NVFP4 weights once.

4 How the work was organized

4.1 People, agents and machines

One person (the author) set goals, approved public submissions and decided rule questions. A lead agent session steered and kept the ledger. Worker sessions each owned one workstream, which we called a lane. Lanes covered attention, dense projections, the expert kernel and its weight stream, a cluster-launch rewrite of the expert kernel, programmatic dependent launch, layer fusions, box efficiency and re-runs of experiments the quota had cut short. Each worked in its own git worktree. On the last afternoon and evening we also ran EAT [6], an open-source supervisor that drives a team of Claude Code agents against a scripted evaluation, with a $400 budget for the layer. It spent $131.94 there and produced the final file's last change.

Official numbers came only from the challenge's judge: `okc check` for private runs and `okc submit` for public ones. The quota started at 10 public runs and 20 checks per problem per day and later rose to 20 and 40. On the last day the judge's backlog stood between 381 and 619 queued jobs. From 3 November we rented the box on RunPod at $5.29 per hour for paired A/B timing against our own public file. Because of its offset from the judge, we used it only to rank two files

against each other, never to predict a judge number.

4.2 Where the code came from

Our own line of files, built from kernels we first wrote for the three standalone component problems, scored 1.168× on its first judged check on 27 October, 2.014× on 29 October and 2.043× (rank 373) on 1 November. On 2 November the tenth-ranked entry stood at 2.422×.

On 1 November the author approved forking the best public layer file, published by the team keell0renz [4]. That file built on alygg77's entry, which in turn built on vjeux's [5]; it also carried code credited to chenwainuo, Matsvei and zhebrak. The challenge terms license every submission under Apache-2.0, and our final file keeps every attribution notice. The unmodified donor measured 2.249× [2.215, 2.283] on our account (job `3a5f879df0d44825`). We never submitted it.

Before using the fork we audited it, mostly its synchronization. Where we could not argue an ordering or progress guarantee from the code, we switched the feature off or replaced it. The audit turned off layer-wide programmatic dependent launch, the expert kernel's dependent launch, the per-expert completion counters that let the expert kernel skip a grid barrier, a data-flag hand-off and two smaller paths. It replaced the expert kernel's launch, which had relied on a software grid barrier inside an ordinary launch, with a checked cooperative launch, so that all of its thread blocks (CTAs) are guaranteed to be resident at once, and it added memory fences to the expert kernel's internal hand-offs. It also rounded the attention output and several of the expert kernel's intermediate values to BF16 where the baseline rounds them, and added a fallback for empty routing. The audited fork alone, judged as a control, scored 2.053×. The first public fork added one change of ours to the expert kernel, which retires each stage's activations at the end of the stage, and scored 2.126× (rank 303). The ledger does not price the audit's changes one by one, and it notes that different inputs and machines may account for part of the gap between those two runs.

4.3 Rules the team adopted

The rules asked every experiment to register its hypothesis, control, affected shapes and acceptance threshold before dispatch, and every loss to go into the ruled-out list with its numbers; reopening an entry required new evidence. From the evening of 4 November a box A/B of a small change to the expert, routing or unpooler kernels needed at least four pairs with the judge's launch configuration before it counted as a null. Public runs needed the author's go-ahead. Section 8 gives the failures behind the rules added during the work.

5 What we inherited and what we changed

The final file is `muse_decode_layer/final/decode_layer.py`, one 21,117-line Python file that compiles its CUDA sources during preparation. Table 3 lists the judged steps after the fork. Gains marked "box" are paired differences on the rented H200, either medians of pairs or means against interleaved controls, as the ledger reports them. They are not additive, and most are smaller than the judge's run-to-run spread, a standard deviation of 0.78% over the final file's nine runs.

From the audited fork's 11,018.9 µs to the final mean of 10,054.0 µs is 965 µs. The three PDL rows and the first counters row account for 357.7 µs, about 50% of the 716.5 µs from the first public fork to the best run, or 61% of the 585.4 µs to the mean. Only the first PDL row (219.5 µs, about a third) is clearly larger than single-run noise, so the split is rough. The expert kernel's own

Table 3: Judged steps after the fork. Each row adds to the one above. Single runs vary by about 0.8% (one standard deviation), so adjacent rows within about 100 µs are not separable; the means at the bottom are more reliable than any single row.

step	job	µs	basis
unmodified donor	`3a5f879d`	10,058.9	1 check
audited fork	`74a29e03`	11,018.9	1 public run
+ stage-end activation retirement	`607eb52b`	10,639.4	1 public run
+ PDL at $T = 1, 16, 32$	`fefd7f7b`	10,419.9	1 public run
+ PDL at $T = 8$, four of six edges	`f931abf5`	10,374.6	1 public run
+ PDL at every T, three edge changes (slack-free rebuild)	`117605d0`	10,333.4	1 public run
+ expert-kernel micro changes, $T = 16$ boundary dispatch	`7d03f88d`	10,194.1	1 public run
+ finisher in registers, cached unpooler loads	`caed0290`	10,146.4	1 public run
+ unpooler PDL, $T = 1$ QKV, attention shared memory	`4a2bf539`	10,109.9	1 check
+ completion counters, 17–32-token build	`4f99c1ec`	10,058.2	mean of 2 checks
+ expert-kernel PDL, counters on larger build, hand-off fix		10,105.4	mean of 9
+ in-house $T = 128$ QKV GEMV		10,084.6	mean of 10
+ EAT's boundary exchange (final file)		10,054.0	mean of 9

dependent launch entered in the same row as the counters on the larger build and is not separable on the judge.

5.1 Inherited structure

Most of the final file's design came with the fork. Below 128 tokens, most dense projections run as streaming GEMV kernels (named `sk2` in the source) that feed weights through a ring of shared-memory buffers filled by the Tensor Memory Accelerator (TMA) and tracked by mbarriers. The weights are stored in a lossless 12-bit encoding of BF16; values that do not fit are stored separately and patched in exactly. These kernels already issued their own dependent-launch triggers. Attention is one fused launch that does QK-norm, RoPE, the FP8 cache write, split-KV attention, the merge and the gated RMSNorm. Each split rescales its partial result to a fixed reference exponent and adds it with FP32 atomics (`red.global.add`) into preallocated accumulators, so no split waits for another's running maximum. In the expert kernel's builds for 17 or more tokens, two TMA producer warps feed four WGMMA warpgroups; the smaller builds use eight compute warps and one helper warp. NVFP4 weights decode to BF16 in registers, and a cost table assigns work items (tiles of one expert's weights) to CTAs stream-K. The builds for 17 or more tokens store block scales losslessly in 6 bits where the scales allow it. Preparation times each backend in graph replay and dispatches per token count. The per-expert completion counters (below) were in the donor file too.

5.2 Programmatic dependent launch, edge by edge

Programmatic dependent launch (PDL) lets a kernel start while its predecessor is still running. The consumer executes `griddepcontrol.wait` before it reads the producer's output, and the producer signals early with `griddepcontrol.launch_dependents`. Work that does not depend on the producer, such as loading weights, can run before the wait.

After the audit switched PDL off, we brought it back one kernel boundary, or edge, at a time. Each edge needed a written argument about the producer's publication, the consumer's wait and, for cooperative grids, residency and progress. These are informal arguments from the code and the

PTX memory model, not machine-checked proofs. A 2,000-replay drift guard runs in preparation and can turn PDL off for a whole token count. It replays fixed inputs and compares against the PDL-off output, so it catches errors beyond a tolerance sized for floating-point merge-order noise (2% relative) but not a consumer that reads ahead of its producer: such a read would see the previous replay's identical values. The written arguments carry that case. The enable set itself is fixed at build time. An earlier gate enabled PDL per token count by comparing output drift with PDL on against drift with it off during preparation; that drift is floating-point merge-order noise, and the gate gave four different enable sets across 16 preparation logs of one file on the box. Two edges around attention produced a 6×10^{-3} output drift at 8 queries per request and stayed off at $T = 8$ until a later file re-enabled all six edges behind the guard.

PDL was the largest single gain after the fork (Table 3). In the final file's preparation log, the median paired saving from PDL, weighted 46:16 over the sliding and full layers, is 6.51, 7.82, 7.77 and 6.80 µs at $T = 1, 8, 16, 32$. The expert-kernel edge, with the expert kernel as a dependent of the routing kernel, gained less than the others. On the box's driver a cooperative launch did not overlap its predecessor, so that edge only shortened graph launch latency, from 0.76–0.96 to 0.38–0.50 µs.

Our first PDL attempt, on 28 October in our own line, triggered every kernel at entry and gained nothing measurable at decode 1 (113.1 to 115.4 µs).

5.3 Per-expert completion counters instead of a grid barrier

From 1 to 4 November, the expert kernel ended with a grid-wide barrier between the expert GEMMs and the final pass, which weights and sums each token's expert outputs. The final pass for a token needs only that token's experts to be finished. The donor's alternative does exactly that. Writers release a per-expert completion counter after each work item's epilogue (`red.release.gpu`), and the final pass acquire-polls (`ld.acquire.gpu`) only the counters of its own token's experts. Our audit had forced this path off, because the poll gave up after four million iterations and then read unreduced data.

On 4 November a top-ten team's write-up, relayed by the author, described removing this barrier. Two two-line patches followed. The first re-enabled the counters on the 17–32-token build and removed the poll's cap. The second extended them to the build for larger T. A ruled-out entry written at 13:19 that day called that build's barrier necessary "by construction", because the CTA that finishes a split work item in that build supposedly never released its expert's counter. A re-read of the code reversed it within half an hour. Removing the cap trades a silent wrong answer for a possible hang. The co-residency of a cooperative launch rules out a hang only if every counter target is reachable, and the ledger records that as a remaining risk. No hang appeared in any judged or box run we recorded.

The box measured decode 32 at $-1.75/-1.67$ µs (sliding/full) for the first patch and verify 32 at $-1.59/-1.40$ µs for the second, with numerics bitwise unchanged. On the judge, the first patch's check scored 10,046.1 and then 10,070.4 µs against 10,109.9 µs for its parent, and the decode 32 paired ratios rose from 1.801 to 1.846 (sliding) and 2.063 to 2.194 (full). With the second patch added, two checks drew 10,111.6 and 10,101.5 µs. At this noise level the judge cannot separate the second patch from zero.

The counters had stayed off for three days. One reason is a final-day rule that ranked work by score gain per microsecond on the short loads to ration judge checks. It admitted decode 16 only as a side effect of a short-load change and excluded decode 32 and the verify 16 and 32 loads outright, and that is where the barrier's cost sits.

5.4 Smaller changes of ours

- At $T = 128$ and 256, the CTA that finishes a split work item keeps its own partial sum in registers instead of writing and re-reading it. Box: verify 32 sliding -2.75 µs, faster in 54 of 54 pairs.
- A publication-ordering fix in the expert kernel's hand-off between its two GEMM phases. It was neutral on the score.
- The unpooler runs as a PDL dependent of the expert kernel at $T = 8$, 16 and 32 (box: -0.97 to -2.06 µs), and its post-wait activation loads go through L1 (box: -0.67 µs at decode 1).
- The $T = 1$ QKV projection moved to the narrow GEMV build already used for $T = 2$ to 8 (box: $-1.32/-0.37$ µs).
- At $T = 128$ an in-house GEMV replaced cuBLASLt for QKV, with attention as its PDL dependent. Evidence is mixed. The final file's preparation log measures $-1.31/-1.86$ µs (sliding/full), while two box rounds measured $+0.72$ and $+1.33$ µs at verify 16 sliding and -2.69 and -2.42 µs at verify 16 full.
- Requesting 81,920 B of shared memory puts two decode 16/32 sliding attention CTAs on each SM (box: -1.19 µs at decode 32 sliding). Requesting 118,784 B for the $T = 8$ finish kernel spreads it to one CTA per SM (box: $-0.41/-0.32$ µs).
- EAT's change, the last one, is in the first residual boundary's kernel at $T = 1$, 8 and 16. In that kernel the eight CTAs that handle one token form a cluster and exchange partial sums of squares for the boundary's two RMS norms. Instead of storing into each other's shared memory and calling `cluster.sync()`, each CTA now sends its partials with `st.async` and waits on its own mbarrier until every byte addressed to it has landed. The kernel also fires its PDL trigger at entry. EAT's written argument covered the exchange. The argument for the trigger was completed only after the close: we traced the kernels that can follow the boundary, one $T = 16$ path by inference, and found that they read only weights and preparation-time tables before their own wait (box: -0.3 to -1.0 µs at decode 1, verify 1 and decode 16).

One constraint shaped the last day. Submissions were capped at 1,000,000 bytes, and the file with all of the above except EAT's change came to 1,057,604. We stripped its comments, keeping legal notices and the strings our own source patches search for, which brought it to 915,316; EAT's change was made on the stripped file. For the CUDA source strings the tool checked that every character outside a comment was unchanged, and for the Python that the syntax tree was unchanged.

5.5 Where the time still goes

Table 1 shows the final file at 1.41–2.08 times the floor. By share of profiled GPU time, the expert kernel is the largest kernel in every cell except three. Attention leads at decode 16 and 32 on full layers and is about even with the experts at verify 32 full (179.6 against 175.4 µs). At decode 1 sliding the expert kernel takes 32% of profiled time (22.5 µs). The QKV, o and fused pooler-router GEMVs take 32.5 µs together, and the unpooler another 3.4 µs. Our weakest cell is decode 32 sliding at 1.855×, where the expert kernel takes half of profiled time (73.8 µs). On the verify 16 and 32 sliding cells, cuBLASLt projections still take 55 to 87 µs per cell.

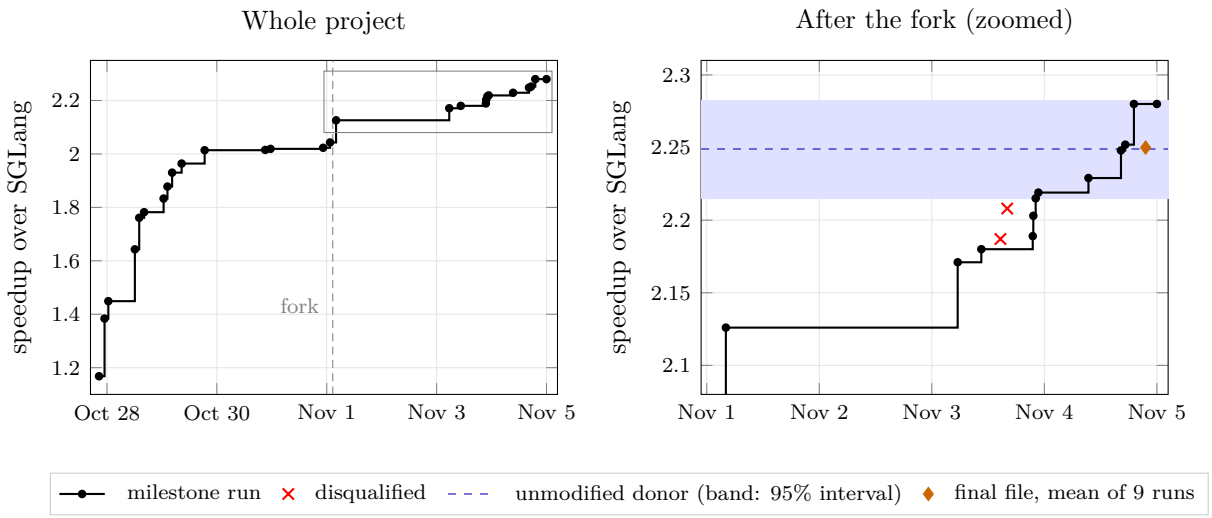


Figure 1: Milestone decode-layer results over time, official judge only. Left: the whole project; before the fork the points are our own line. Right: the boxed region enlarged. The unmodified donor file was measured once on our account (2.249×, shaded 95% interval). The final file's nine runs took place on the evening of 4 November, and their mean falls inside that interval. Before 1 November, runs without a judge timestamp in the repository are placed at the time the ledger recorded them, minutes to hours after the run; the second disqualified run is placed at its submission time.

5.6 A review of the final file after the close

After the close we reviewed the final file's newest code: EAT's change, the $T = 128$ QKV GEMV, the cluster boundary kernels with the kernels that can follow them, and the Python preparation and dispatch layers [7]. The attention and expert CUDA sources and most other projection kernels were not part of it. None of its findings concerns the arithmetic on the scored shapes; they concern checks that run during preparation, fallback and timeout paths, and the file's documentation. The PDL and $T = 128$ drift guards replay each input back to back, so they cannot see a read that runs ahead of its producer. The sentinel words after each scratch buffer were checked before most of the preparation-time replays ran. Only the $T = 8$ PDL stage checked elapsed time, against its own 230 s budget; nothing checked preparation as a whole against the 300 s limit. The $T = 128$ GEMV ignores a timed-out mbarrier wait. A successor file in the repository adds fresh-input stress rounds, a final sentinel check and a preparation budget. It has not been run on a GPU or judged.

6 Results

Figure 1 shows the milestones among the judged layer results, plus the two disqualified runs. Before the fork they are selected new bests of our own line; after it they are the public board rows, so a private check that reached a lower time before its public run does not appear. Table 4 in the appendix lists them with job ids. The score rose in two phases. Our own line went from 1.168× to 2.043× in four days, mostly by porting our standalone kernels into the layer: the fused W4A16 expert kernel, the one-launch attention and the GEMV projections. The fork started at 2.126×, and three days of work on it took the best run to 2.280× and the mean to 2.250×.

The final file ran nine times on the judge, three public runs and six private checks. They span

9,922.9 to 10,160.8 µs, with a mean of 10,054.0 µs (2.250×) and a standard deviation of 0.78%. The best of them, run `908cc07e43364008`, is the board row: 2.2796× with a 95% interval of [2.2454, 2.3138]. The judge returns a cached result for identical bytes, so each repeat differed only in a nonce. The best run is 1.3% faster than the mean. That selection effect is part of the board position, and entries are better compared by their means.

A later EAT file with several further changes had a slightly lower eight-run mean (10,029.6 µs), but none of its runs beat the row. Board ranks fall as other teams submit, so each rank below is paired with its time. At 22:36 Pacific on 4 November the row ranked 153rd of the layer board, which had had about 1,220 entries on 2 November. At 23:40, the last reading before the close, the team stood 158th on the overall board. The last recorded score of the 100th-ranked entry, at 18:28, was 2.3028×.

Per load (Table 1), the best run's speedups are 2.574 (decode 1), 2.134 (decode 16), 1.967 (decode 32), 2.569 (verify 1), 2.169 (verify 16) and 2.330 (verify 32). Their geometric mean is 2.2796, the reported score.

The standalone boards, ranked separately, ended at 2.3893× for the expert kernel (rank 258), 2.6043× for attention and 1.5321× for the dense projections.

6.1 Threats to validity

- The donor was measured once, and we compare the final file's nine-run mean against that one run's interval. A second measurement could move it by about 1%.
- Box results carry no reported dispersion in this paper and are mostly smaller than the judge's run-to-run spread. They ranked candidates; they do not establish judge gains.
- Steps in Table 3 are mostly single runs, and attributions between adjacent rows are within noise.
- The best run is selected from nine.
- Everything here is one layer, one model, one GPU type and one judge.

7 What did not work

These are the most instructive entries in the ruled-out list, with the numbers that closed them.

L2 prefetch of later weights during attention. Prefetching the 34 MB of o, router and unpooler weights beside attention cut the o GEMV from 7.3 to 5.9 µs, but the whole step got 3.5–4.1 µs slower at decode 1 between two judge checks. The prefetch traffic cost more than the hits saved. Several other teams reported the same in public.

Fusing across stage boundaries. One cooperative launch for the o projection, the attention boundary and the pooler/router saved 0.5–1.3 µs inside the process and nothing on any short load. Fusing the experts into the unpooler made decode 1 worse (80.7 against 76.8 µs); it replaced two launch boundaries with five dependent global round trips. A cooperative o plus boundary kernel was 1.16 µs slower in 9 of 9 rounds. Our hypothesis, not a measurement, is that each grid barrier costs about 1 µs.

Using idle SMs. The QKV GEMV launches 96 CTAs on 132 SMs. A stream-K version with 128 one-wave CTAs at $T = 8$–32 lost 2.9 to 4.0 µs to the join alone, and the variant with no reduction lost too. Idle SMs were not what limited this kernel.

More concurrency in the expert kernel. Two expert CTAs per SM lost 7.9/8.5 µs at decode 1, through a doubled per-CTA prologue, grid barrier and final work. A four-slot weight ring lost 2.0/2.2 µs. A second N-tile, meant to avoid re-streaming weights at decode 16, lost about 10 µs, the third loss on that idea. In a math-only measurement at $T = 8$, the expert kernel's inner loop sat at an issue-throughput ceiling of about 0.5 warp-instructions per cycle per SM sub-partition, the same at four and six consumer warps. If that holds in the full kernel, the next gain there at small T needs fewer instructions per weight word.

Synthetic routing. We first chose expert-kernel variants on synthetic routing, where a 1–3 µs margin on two seeds decided a 30 µs question. With routing skewed the way the judge's is, verify 16 dropped 25–30 µs.

Porting layer wins back to the standalone problems. Projection and attention choices tuned inside the layer gave nothing for the standalone problems on the box, so we never submitted them. Only the expert-kernel port helped there, by about 0.7% on the box. We did not find out why.

7.1 The routing-slack disqualification

The rules let routing differ from the reference on near-ties. Each chosen expert has to score within 0.05 of the reference's eighth best, and clear winners must be kept [2]. On this GPU each distinct local expert costs the fused kernel about 6.3 µs of weight reads. So at $T = 1$ we swapped near-tied local experts for near-tied experts on other GPUs, which this GPU then never reads. Decode 1 got about 5 and 3 µs faster on sliding and full layers. The file passed every correctness test. The author approved it under the written tolerances, and it went public at 10,343.1 µs, rank 355.

By 17:45 Pacific on 3 November the organizers had disqualified two public runs of the technique, and we withdrew the other two. Their reason was that the change "swaps near-tied experts for ones on other GPUs so their weights are never read, which changes the model's output" and so "goes against the spirit of the challenge". By 21:32, after the withdrawals, the row was a slack-free rebuild at 10,333.4 µs (2.189×, rank 367). Every file built on the slack file had to be rebuilt from its slack-free parent and re-qualified. A review confirmed that the routing itself was restored, beyond checking that the slack code was gone. Our release audit had also counted a disqualified source as clean, because it read the numerical pass flags and not the separate disqualification fields. A tolerance written to absorb a reference's rounding is not permission to choose outputs for speed, and passing the checks is not the same as being eligible.

8 Process lessons

Each failure below cost real time and ended in a written rule. Some would trouble any team; the first and the last two follow from how agent sessions work.

Estimated clocks. On the last day the lead agent stamped ledger times from its own estimates and ran 3.5 hours ahead of the real clock. In the same incident, a plan written that morning said 11.5 hours remained when about 13.5 did. Rule: every ledger or schedule time comes from `date` in the same command.

Shared checkouts. Another session's merge reset the working tree and wiped staged edits. The newly added files came back from dangling objects (`git fsck --dangling`); the other edits were lost. A union merge of a CUDA source doubled a pipeline stage-advance line, and that line was pushed. Rules: edit, stage, commit and push in one call; resolve source conflicts by hand; give every branch its own worktree.

A flag that changed the experiment. A box script's `--skip-pdl8-gate` flag also turned off the expert kernel's PDL edge. With the flag, one change read faster in 7 of 7 pairs; without it, it read null over 3 pairs. We built the next file without that change. Four pairs with the edge on later showed it 0.48/0.52 µs faster. Rule: measure with the configuration the judge uses, in at least four pairs.

Rationing that narrowed the search. The rule that ranked work by gain per microsecond on the short loads was meant to ration judge checks. It also narrowed what the agents looked at, and the completion-counter fix sat in the excluded cells. Rule: rank engineering work by its effect on the whole score, and treat a feature compiled in but switched off as a lead to re-check.

Context limits. Lane sessions reached 250,000–500,000 tokens per round and became slower, costlier and less reliable. Rule: each lane writes a checkpoint file after every step and restarts fresh after about 60 tool calls.

An async judge. With the judge queue slow on the last days, a session that waited on a job did nothing else. Rule: submit with `--no-wait`, record the job id in the ledger and wait in the background.

9 Discussion

The fork's inherited design carried the score. Our work after the fork took the file from 2.126× to a mean of 2.250×, and against the unmodified donor measured on our account that gain is zero within the judge's interval. What the work bought is the same speed with written arguments for ordering and progress on every dependent-launch edge and on the completion counters, which the donor shipped without. The argument for the last edge, EAT's entry trigger, was finished only after the close, and the guards that run in preparation cannot catch an ordering error on their own. We did not show that the donor was wrong anywhere. Whether those arguments are worth the time is a fair question for anyone running a short competition.

Two observations may help others working at small batch on this GPU. First, none of the cross-stage fusions we tried beat a PDL kernel boundary inside a CUDA graph, and two were measurably slower. Fusion inside a stage, such as the one-launch attention kernel, still paid. Second, a feature that someone switched off for safety deserves a re-check against the code once the pressure is gone.

For agent-driven kernel work, the largest gain came from reading a public file, and the largest later step from re-reading code paths the audit had disabled. Local tuning gave many small gains, most of them within the judge's noise. Measurement discipline mattered most where it was weakest. A box script measured a different configuration than the judge, a quota rule kept a disabled fix out of view for three days, and a numerical tolerance was read as permission.

Use of AI tools

LLM coding agents wrote most of the code and kept the experiment records: Anthropic's Claude models through Claude Code, OpenAI Codex sessions, and the EAT supervisor running Claude Code agents. A Claude agent drafted this report from the repository's ledgers after the close, and further agents checked its claims against the sources. The author chose the work, approved every public submission directly or through standing authorizations, and is responsible for the content.

References

[1] Muse Spark Open Kernel Challenge. `https://openkernelchallenge.dev`, rules at `https://openkernelchallenge.dev/rules`, 2026. Accessed November 2026.

[2] Muse Spark Open Kernel Challenge. Problem statement for `muse_decode_layer`, `TASK.md` written by `okc init` 0.7.2, 2026. A copy is in [3].

[3] The repository for this work, with run ledgers, profiles and the ruled-out list. `https://github.com/yelgabo/okc`, 2026.

[4] keell0renz. Muse open kernel challenge entry and `LESSONS.md`. `https://github.com/keell0renz/muse-open-kernel-challenge`, 2026.

[5] vjeux. OKC 2026 submissions. `https://github.com/vjeux/okc-2026`, 2026.

[6] caoshiyi. EAT. `https://github.com/caoshiyi/EAT`, 2026.

[7] Review of the final decode-layer file, `docs/decode-layer-final-review.md` in [3], 2026.

[8] L. Zheng, L. Yin, et al. SGLang: Efficient execution of structured language model programs. arXiv:2312.07104, 2023.

[9] J. Shah, G. Bikshandi, et al. FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. arXiv:2407.08608, 2024.

[10] E. Frantar, R. L. Castro, et al. MARLIN: Mixed-precision auto-regressive parallel inference on large language models. arXiv:2408.11743, 2024.

[11] M. Osama, D. Merrill, et al. Stream-K: Work-centric parallel decomposition for dense matrix-matrix multiplication on the GPU. arXiv:2301.03598, 2023.

[12] T. Gale, D. Narayanan, et al. MegaBlocks: Efficient sparse training with mixture-of-experts. arXiv:2211.15841, 2022.

[13] DeepSeek-AI. DeepGEMM. `https://github.com/deepseek-ai/DeepGEMM`, 2025.

[14] A. Ouyang, S. Guo, et al. KernelBench: Can LLMs write efficient GPU kernels? arXiv:2502.10517, 2025.

[15] NVIDIA. Parallel Thread Execution ISA, memory consistency model. `https://docs.nvidia.com/cuda/parallel-thread-execution/`. Accessed November 2026.

[16] NVIDIA. CUDA Programming Guide, programmatic dependent launch and synchronization. `https://docs.nvidia.com/cuda/cuda-programming-guide/`. Accessed November 2026.

A Judged layer runs

Table 4: Decode-layer milestone results, plus the disqualified runs. Some intermediate bests are omitted, among them private checks that preceded a public run of the same change. Ranks are as of when the row was set. Times are the judge's, which equal SGLang's fixed 22,620.3 µs over the speedup. Before 1 November a time of day is given only where a judge receipt records it; other rows give the Pacific date of the ledger entry. "check" is a private run; DQ means disqualified. Sources: `docs/runs-decode-layer.md` and `docs/steering/2026-11-02-final-day-directive.md` in [3].

date (PT)	job	change	µs	speedup	rank
Oct 27	`c61c26ce`	MoE port only (check)	19,367.7	1.168	
Oct 27	`6c9eb0ed`	+ W4A16 expert GEMMs (check)	16,341.0	1.384	
Oct 28	`b216cf4a`	cluster residual kernel (check)	15,609.5	1.449	
Oct 28	`f23f1c81`	fused MoE, GEMV projections	13,769.9	1.643	
Oct 28	`cc7a07e2`	one-launch attention	12,847.9	1.761	
Oct 28 16:13	`a41ace0b`	MoE ordering repair	12,692.1	1.782	
Oct 29	`344ae513`	MoE configs chosen on judge-like routing	12,338.0	1.833	
Oct 29	`5b95c24e`	attention trimming	12,046.1	1.878	
Oct 29	`0fae6468`	another MoE work schedule	11,720.2	1.930	
Oct 29	`9b921feb`	routing front end	11,517.0	1.964	
Oct 29	`3fbb3ad0`	clean release, MoE schedule	11,229.9	2.014	
Oct 30 21:06	`0f58d5d8`	expert-kernel mode and stable tuner	11,227.0	2.015	
Oct 30 23:26	`7b84ed43`	projection plan, routing side stream	11,205.4	2.019	
Oct 31 22:25	`8f2780d7`	attention port	11,182.3	2.023	
Nov 1 01:27	`cb71acec`	own short-T boundary, cuBLASLt plans	11,069.7	2.043	373
Nov 1 04:02	`607eb52b`	audited fork + stage-end retirement	10,639.4	2.126	303
Nov 3 05:25	`fefd7f7b`	PDL at $T = 1, 16, 32$	10,419.9	2.171	369
Nov 3 10:40	`f931abf5`	PDL at $T = 8$	10,374.6	2.180	364
Nov 3 14:40	`9d80ae24`	routing slack (DQ)	10,343.1	2.187	355
Nov 3 16:07	`86ed5dd3`	routing slack repeat (DQ)	10,243.9	2.208	
Nov 3 21:32	`117605d0`	slack-free rebuild, PDL at every T	10,333.4	2.189	367
Nov 3 21:39	`a7741ba2`	expert-kernel micro changes	10,267.5	2.203	353
Nov 3 22:09	`1798d468`	$T = 16$ residual on a per-row kernel	10,211.3	2.215	322
Nov 3 22:43	`7d03f88d`	follow-up to the previous row	10,194.1	2.219	311
Nov 4 09:25	`caed0290`	finisher in registers, cached loads	10,146.4	2.229	308
Nov 4 16:20	`40ac6a24`	expert kernel as PDL dependent	10,061.9	2.248	
Nov 4 17:16	`5dbfa4b0`	counters on both builds, hand-off fix	10,044.3	2.252	246
Nov 4 19:07	`908cc07e`	$T = 128$ QKV GEMV + EAT exchange (final)	9,922.9	2.280	146